4 Compiler Design (52)

4.0.1 NIELIT 2018-62

Among all given option, ____ must reside in the main memory.

A. Assembler B. Compiler C. Linker D. Loader

nielit-2018 compiler-design easy

Answer key

4.0.2 NIELIT 2016 DEC Scientist B (CS) - Section B: 36

The graph that shows basic blocks and their successor relationship is called:

A. DAG B. Control graph C. Flow graph D. Hamiltonian graph

nielit2016dec-scientistb-cs compiler-design directed-acyclic-graph

Answer key

4.0.3 NIELIT 2016 DEC Scientist B (CS) - Section B: 22

The structure or format of data is called

A. Syntax B. Struct C. Semantic D. none of the above

nielit2016dec-scientistb-cs compiler-design easy

Answer key

4.0.4 NIELIT 2016 MAR Scientist C - Section C: 10

Relocation bits used by relocating loader are specified by

A. Relocating loader itself
B. Linker
C. Assembler
D. Macro processor

nielit2016mar-scientistc compiler-design

Answer key

4.0.5 NIELIT 2016 MAR Scientist C - Section C: 9

Which of the following can be accessed by transfer vector approach of linking?

A. External data segments
B. External subroutines
C. Data located in other procedure
D. All of these

nielit2016mar-scientistc compiler-design

Answer key

4.1 Assembler (1)

4.1.1 Assembler: NIELIT 2016 MAR Scientist B - Section C: 31

In a single pass assembler, most of the forward references can be avoided by putting the restriction

A. on the number of strings/lifereacts.
B. that the data segment must be defined after the code segment.
C. on unconditional rump.
D. that the data segment be defined before the code segment.

nielit2016mar-scientistb compiler-design assembler

Answer key

4.2 Assembly (1)

4.2.1 Assembly: NIELIT 2021 Dec Scientist B - Section B: 50

Assembly language _____.

A. uses mnemonics or alphabetic codes in place of binary numbers used in machine language
B. is the easiest language to write programs
C. need not be translated into machine language
D. is high level language.

nielit-2021-it-dec-scientistb assembly programming-in-c co-and-architecture

Answer key

4.3 Code Optimization (9)

4.3.1 Code Optimization: NIELIT 2017 DEC Scientist B - Section B: 42

The optimization phase in a compiler generally

A. Reduces the space of the code
B. Optimizes the code to reduce execution time
C. Both (A) and (B)
D. Neither (A) nor (B)

nielit2017dec-scientistb compiler-design code-optimization

Answer key

4.3.2 Code Optimization: NIELIT 2017 DEC Scientist B - Section B: 5

Which of the following is machine independent optimization?

A. Loop optimization
B. Redundancy Elimination
C. Folding
D. All of the option

nielit2017dec-scientistb compiler-design code-optimization

Answer key

4.3.3 Code Optimization: NIELIT 2018-64

Which of the following code replacements is an example of operator strength reduction?

A. Replace $\mathrm{P\hat{}2}$ by P^*P
B. Replace P^*16 by $P << 4$
C. Replace $\mathrm{pow}(P,3)$ by P^*P^*P
D. Replace $(P << 5) - P$ by P^*3

nielit-2018 compiler-design code-optimization

Answer key

4.3.4 Code Optimization: NIELIT 2018-77

_____ merges the bodies of two loops

A. loop rolling
B. loop folding
C. loop merge
D. loop jamming

nielit-2018 compiler-design code-optimization easy

Answer key

4.3.5 Code Optimization: NIELIT 2022 April Scientist B | Section B | Question: 52

Consider the expression $(a-1) * (((b+c)/3)) + d))$. Let X be the minimum number of registers required by an optimal code generation (without any register spill) algorithm for a load/store architecture, in which

i. only load and store instructions can have memory operands and
ii. arithmetic instructions can have only register or immediate operands.

The value of X is _____ .

A. 2
B. 1
C. 4
D. 3

nielit2022apr-scientistb compiler-design code-optimization co-and-architecture

4.3.6 Code Optimization: NIELIT 2022 April Scientist B | Section B | Question: 56

Consider these two functions and two statements $\mathrm{S1}$ and $\mathrm{S2}$ about them.

```
int work1(int *a, int i, int j)
{
int x = a[i+2];
a[j] = x+1;
return a[i+2] - 3;
}
```

```
int work2(int *a, int i, int j)
{
int t1 = i+2;
int t2 = a[t1];
a[j] = t2+1;
return t2 - 3;
}
```

$\mathrm{S1}$: The transformation form work1 to work2 is valid, i.e., for any program state and input arguments, work2 will compute the same output and have the same effect on program state as work1

$\mathrm{S2}$: All the transformations applied to work1 to get work2 will always improve the performance (i.e reduce CPU time) of work2 compared to work1

A. $S1$ is false and $S2$ is false
B. $S1$ is false and $S2$ is true
C. $S1$ is true and $S2$ is false
D. $S1$ is true and $S2$ is true

nielit2022apr-scientistb programming-in-c code-optimization data-structures

4.3.7 Code Optimization: NIELIT Scientific Assistant A 2020 November: 50

Some code optimizations are carried out on the intermediate code because:

A. they enhance the portability of the compiler to other target processors
B. program analysis is more accurate on intermediate code than on machine code
C. the information from data flow analysis cannot otherwise be used for optimization
D. the information from the front end cannot otherwise be used for optimization

nielit-sta-2020 compiler-design code-optimization normal

Answer key

4.3.8 Code Optimization: NIELIT Scientific Assistant A 2020 November: 53

Which of the following is not true for tree and graph?

A. A tree is a graph
B. A graph is a tree
C. Tree can have a cycle
D. Tree is a DAG

nielit-sta-2020 compiler-design code-optimization directed-acyclic-graph easy

Answer key

4.3.9 Code Optimization: NIELIT Scientific Assistant A 2020 November: 66

Peephole optimization is a :

A. Loop optimization
B. Local optimization
C. Constant folding
D. Data flow analysis

nielit-sta-2020 compiler-design code-optimization easy

Answer key

4.4 Compilation Phases (1)

4.4.1 Compilation Phases: NIELIT Scientific Assistant A 2020 November: 67

Which one of the following statements is $FALSE$?

A. Context-free grammar can be used to specify both lexical and syntax rules.
B. Type checking is done before parsing
C. High level language programs can be translated to different Intermediate Representations
D. Arguments to a function can be passed using the program stack

nielit-sta-2020 compiler-design easy compilation-phases

Answer key

4.5 Compilations (1)

4.5.1 Compilations: NIELIT 2017 DEC Scientist B - Section B: 26

The grammar $S \rightarrow aSb \mid bSa \mid SS \mid \varepsilon$ is:

A. Unambiguous CFG
B. Ambiguous CFG
C. Not a CFG
D. Deterministic CFG

nielit2017dec-scientistb compiler-design compilations context-free-grammar ambiguous

Answer key

4.6 Compiler tokenization (2)

4.6.1 Compiler tokenization: NIELIT 2018-56

Identify the total number of tokens in the given statement

```
printf("A%B=",&i);
```

A. 7 B. 8 C. 9 D. 13

nielit-2018 compiler-design compiler-tokenization easy

Answer key

4.6.2 Compiler tokenization: NIELIT Scientific Assistant A 2020 November: 100

The number of tokens in the following C/C++ statement is :

```
printf("i=%d, &i=%xx", i&i);
```

A. 9 B. 6 C. 10 D. 12

nielit-sta-2020 compiler-design lexical-analysis compiler-tokenization easy

Answer key

4.7 Context Free Grammar (2)

4.7.1 Context Free Grammar: NIELIT 2018-32

The context free grammar $S \rightarrow aSb \mid bSa \mid \epsilon$ generates:

A. Equal number of a's and b's
B. Unequal number of a's and b's
C. Any number of a's followed by any number of b's
D. Any number of b's followed by any number of a's

nielit-2018 compiler-design context-free-grammar

Answer key

4.7.2 Context Free Grammar: NIELIT 2021 Dec Scientist A - Section B: 92

Consider the grammar with nonterminals $\mathrm{N} = \{ \mathrm{S}, \mathrm{C}, \mathrm{S}_1 \}$ terminals $\mathrm{T} = \{ \mathrm{a}, \mathrm{b}, \mathrm{i}, \mathrm{t}, \mathrm{e} \}$ With S as the start symbol, and the following set of rules :

$\mathrm{S} \rightarrow \mathrm{i}\,\mathrm{Ct}\,\mathrm{SS}_1 | \mathrm{a}$

$\mathrm{S}_1 \rightarrow \mathrm{eS} | \in$

$\mathrm{C} \rightarrow \mathrm{b}$

The grammar is not $\mathrm{LL}(1)$ because :

A. It is left recursive
B. It is right recursive
C. It is ambiguous
D. It is not context free

nielit2021dec-scientista grammar compiler-design parsing context-free-grammar

Answer key

4.8 Data Flow Analysis (1)

4.8.1 Data Flow Analysis: NIELIT 2021 Dec Scientist B - Section B: 117

A variable p is said to be line at point m if and only if **:**

A. p is assigned at point m
B. p is not assigned at point m
C. Value of p at m would be used along some path in the flow graph starting at point m
D. Value of p at m would be used along some path in the flow graph ending at point m

nielit2021dec-scientistb compiler-design data-flow-analysis

4.9 Dynamic Programming (1)

4.9.1 Dynamic Programming: NIELIT Scientific Assistant A 2020 November: 52

In case of the dynamic programming approach the value of an optimal solution is computed in :

A. Top down fashion
B. Bottom up fashion
C. Left to Right fashion
D. Right to Left fashion

nielit-sta-2020 compiler-design dynamic-programming

Answer key

4.10 First and Follow (1)

4.10.1 First and Follow: NIELIT 2017 OCT Scientific Assistant A (CS) - Section B: 10

Consider an ε-tree CFG. If for every pair of productions $A \rightarrow u$ and $A \rightarrow v$

A. If $\mathrm{FIRST(u)} \cap \mathrm{FIRST(v)}$ is empty then the CFG has to be $LL(1)$.
B. If the CFG is $LL(1)$ then $\mathrm{FIRST(u)} \cap \mathrm{FIRST(v)}$ has to be empty.

C. Both (A) and (B)
D. None of the above

nielit2017oct-assistanta-cs compiler-design context-free-grammar first-and-follow

Answer key

4.11 Grammar (1)

4.11.1 Grammar: NIELIT 2017 July Scientist B (CS) - Section B: 47

What is the maximum number of reduce moves that can be taken by a bottom-up parser for a grammar with no epsilon and unit production (i.e.,of type $A \rightarrow \epsilon$ and $A \rightarrow a$) to parse a string with n tokens?

A. $n/2$ B. $n-1$ C. $2n-1$ D. 2^n

nielit2017july-scientistb-cs compiler-design grammar

Answer key

4.12 Intermediate Code (2)

4.12.1 Intermediate Code: NIELIT 2018-76

Identify the correct nodes and edges in the given intermediate code

1. $i = 1$
2. $t1 = 5 * i$
3. $t2 = 4 * t1$
4. $t3 = t2$
5. $a[t3] = 0$
6. $i = i + 1$
7. if $i < 15$ goto(2)

A. 33 B. 44 C. 43 D. 34

nielit-2018 compiler-design intermediate-code

Answer key

4.12.2 Intermediate Code: NIELIT 2021 Dec Scientist A - Section B: 54

One of the purposes of using intermediate code in compilers is to:

A. make parsing and semantic analysis simpler
B. improve error recovery and error reporting
C. increase the chances of reusing the machine – independent code optimizer in other compilers
D. improve the register allocation

nielit2021dec-scientista compiler-design intermediate-code

Answer key

4.13 Interpreter (1)

4.13.1 Interpreter: NIELIT 2017 DEC Scientist B - Section B: 54

Which of the following statements is/are true in the context of interpreters?

- $S1$: Interpreters process program according to the logical flow of control through the program.
- $S2$: Interpreter translates and executes the error-free first instruction before it goes to the second.
- $S3$: Interpreter processing time is less compared with compiler.
- $S4$: LISP and Prolog are interpreted languages.

A. Only $S1$
B. Only $S3$
C. Only $S1$, $S2$ and $S3$
D. Only $S1$, $S2$ and $S4$

nielit2017dec-scientistb compiler-design interpreter

Answer key

4.14 Is&software Engineering (1)

4.14.1 Is&software Engineering: NIELIT Scientist B 2020 November: 81

Debugger is a program that:

A. Allows to examine and modify the contents of registers
B. Allows to set breakpoints, execute a segment of program and display contents of register
C. Does not allow execution of a segment of program
D. All the options

nielit-scb-2020 operating-system is&software-engineering

Answer key

4.15 LR Parser (3)

4.15.1 LR Parser: NIELIT 2017 DEC Scientist B - Section B: 33

Which of the following statements is/are false?

$S1$: $LR(0)$ grammar and $SLR(1)$ grammar are equivalent

$S2$: $LR(1)$ grammar are subset of $LALR(1)$ grammars

A. $S1$ only B. $S1$ and $S2$ both C. $S2$ only D. None of the options

nielit2017dec-scientistb compiler-design grammar lr-parser easy

Answer key

4.15.2 LR Parser: NIELIT 2022 April Scientist B | Section B | Question: 64

Which of the following statements about the parser is/are correct?

I. Canonical LR is more powerful than SLR.
II. SLR is more powerful than LALR.
III. SLR is more powerful than canonical LR.

A. (I) only
B. (II) only
C. (III) only
D. (II) and (III) only

nielit2022apr-scientistb parsing compiler-design lr-parser

Answer key

4.15.3 LR Parser: NIELIT Scientific Assistant A 2020 November: 94

Shift reduce parsing can also be called as:

A. Reverse of the Right Most Derivation
B. Right Most Derivation
C. Left Most Derivation
D. None of the options

nielit-sta-2020 compiler-design parsing lr-parser easy

Answer key

4.16 Lexical Analysis (3)

4.16.1 Lexical Analysis: NIELIT 2016 DEC Scientist B (CS) - Section B: 57

The output of lexical analyzer is:

A. A set of regular expressions
B. Strings of character
C. Syntax tree
D. Set of tokens

nielit2016dec-scientistb-cs compiler-design lexical-analysis easy

Answer key

4.16.2 Lexical Analysis: NIELIT 2017 July Scientist B (CS) - Section B: 49

In a compiler, keywords of a language are recognized during

A. parsing of the program
B. the code generation
C. the lexical analysis of the program
D. dataflow analysis

nielit2017july-scientistb-cs compiler-design lexical-analysis easy

Answer key

4.16.3 Lexical Analysis: NIELIT 2021 Dec Scientist B - Section B: 99

Two Important lexical categories are _____ .

A. White Space
B. Comments

C. White Space & Comments
D. None of the options

nielit2021dec-scientistb compiler-design lexical-analysis

Answer key

4.17 Linker (2)

4.17.1 Linker: NIELIT 2016 MAR Scientist B - Section C: 32

A linker is given object module for a set of programs that were compiled separately. What information need not be included in an object module?

A. Object mode
B. Relocation bits
C. Names and locations of all external symbols defined in the object module.
D. Absolute addresses of internal symbols.

nielit2016mar-scientistb compiler-design linker

Answer key

4.17.2 Linker: NIELIT 2017 July Scientist B (CS) - Section B: 57

A system program that combines the separately complied modules of a program into a form suitable for execution

A. assembler B. linking loader C. cross compiler D. load and go

nielit2017july-scientistb-cs compiler-design linker

Answer key

4.18 Parsing (8)

4.18.1 Parsing: NIELIT 2016 DEC Scientist B (CS) - Section B: 40

A top down parser generates

A. Left most derivation
B. Right most derivation
C. Left most derivation in reverse
D. Right most derivation in reverse

nielit2016dec-scientistb-cs compiler-design parsing easy

Answer key

4.18.2 Parsing: NIELIT 2016 MAR Scientist C - Section C: 11

The most powerful parser is

A. SLR B. LALR C. Canonical LR D. Operator-precedence

nielit2016mar-scientistc compiler-design parsing easy

Answer key

4.18.3 Parsing: NIELIT 2016 MAR Scientist C - Section C: 12

YACC builds up

A. SLR parsing table
B. Canonical LR parsing table
C. LALR parsing table
D. None of these

nielit2016mar-scientistc compiler-design parsing easy

Answer key

4.18.4 Parsing: NIELIT 2016 MAR Scientist C - Section C: 13

Context-free grammar can be recognized by

A. finite state automation
B. 2- way linear bounded automata
C. push down automata
D. both (B) and (C)

nielit2016mar-scientistc compiler-design parsing

Answer key

4.18.5 Parsing: NIELIT 2021 Dec Scientist B - Section B: 70

A bottom-up parser generates _____ .

A. Rightmost derivation
B. Rightmost derivation in reverse
C. Leftmost derivation
D. Leftmost derivation in reverse

nielit2021dec-scientistb compiler-design parsing

4.18.6 Parsing: NIELIT Scientific Assistant A 2020 November: 118

The $\mathrm{LL}(1)$ and $\mathrm{LR}(0)$ techniques are _____

A. Both same in power
B. Both simulate reverse of right most derivation
C. Both simulate reverse of left most derivation
D. Incomparable

nielit-sta-2020 compiler-design parsing

Answer key

4.18.7 Parsing: NIELIT Scientific Assistant A 2020 November: 46

Assume that the SLR parser for a grammar G has n_1 states and the LALR parser for G has n_2 states. The relationship between n_1 and n_2 is :

A. n_1 is necessarily less than n_2
B. n_1 is necessarily equal to n_2
C. n_1 is necessarily greater than n_2
D. none of the options

nielit-sta-2020 compiler-design parsing

Answer key

4.18.8 Parsing: NIELIT Scientific Assistant A 2020 November: 69

Type of conflicts that can arise in $\mathrm{LR}(0)$ techniques are _____.

A. Shift-reduce conflict
B. Shift-Shift conflict
C. Both “Shift-reduce conflict” & “Shift-Shift conflict”
D. None of the options

nielit-sta-2020 compiler-design parsing easy

Answer key

4.19 Runtime Environment (1)

4.19.1 Runtime Environment: NIELIT 2016 DEC Scientist B (CS) - Section B: 14

The identification of common sub-expression and replacement of run time computations by compile-time computations is:

A. Local optimization
B. Constant folding
C. Loop Optimization
D. Data flow analysis

nielit2016dec-scientistb-cs compiler-design runtime-environment

Answer key

4.20 Stack (1)

4.20.1 Stack: NIELIT 2022 April Scientist B | Section B | Question: 103

Which of the following are true?

I. A programming language which does not permit global variables of any kind and has no nesting of procedures/functions, but permits recursion can be implemented with static storage allocation
II. Multi-level access link (or display) arrangement is needed to arrange activation records only if the programming language being implemented has nesting of procedures/functions
III. Recursion in programming languages cannot be implemented with dynamic storage allocation
IV. Nesting procedures/functions and recursion require a dynamic heap allocation scheme and cannot be implemented with a stack-based allocation scheme for activation records
V. Programming languages which permit a function to return a function as its result cannot be implemented with a stack-based storage allocation scheme for activation records

A. (II) and (V) only
B. $(\mathrm{I}), (\mathrm{III})$ and (IV) only
C. $(\mathrm{I}), (\mathrm{II})$ and (V) only
D. $(\mathrm{II}), (\mathrm{III})$ and (V) only

nielit2022apr-scientistb compiler-design operating-system stack memory-management programming-in-c

4.21 Syntax Directed Translation (3)

4.21.1 Syntax Directed Translation: NIELIT 2016 DEC Scientist B (CS) - Section B: 43

Syntax directed translation scheme is desirable because

A. It is based on the syntax
B. It is easy to modify
C. Its description is independent of any implementation
D. All of these

nielit2016dec-scientistb-cs compiler-design syntax-directed-translation

Answer key

4.21.2 Syntax Directed Translation: NIELIT 2017 OCT Scientific Assistant A (CS) - Section B: 11

Synthesized attribute can easily be simulated by an

A. LL grammar
B. ambiguous grammar
C. LR grammar
D. none of the above

nielit2017oct-assistanta-cs compiler-design syntax-directed-translation

Answer key

4.21.3 Syntax Directed Translation: NIELIT 2022 April Scientist B | Section B | Question: 69

Consider the following Syntax Directed Translation Scheme (SDTS), with non-terminals $\{\mathrm{S},\mathrm{A}\}$ and terminals $\{\mathrm{a},\mathrm{b}\}$.

- $\mathrm{S} \rightarrow \mathrm{aA} \quad \{\text{print } 1\}$
- $\mathrm{S} \rightarrow \mathrm{a} \quad \{\text{print } 2\}$
- $\mathrm{A} \rightarrow \mathrm{Sb} \quad \{\text{print } 3\}$

Using the above SDTS, the output printed by a bottom-up parser, for the input aab is:

A. $1\,3\,2$
B. $2\,2\,3$
C. $2\,3\,1$
D. Syntax Error

nielit2022apr-scientistb syntax-directed-translation parsing compiler-design

Answer key

4.22 Three Address Code (1)

4.22.1 Three Address Code: NIELIT 2021 Dec Scientist B - Section B: 105

A sequence of statement of the form $x = y \text{ op } z$ is called a **:**

A. Three address code
B. Syntax tree
C. Postfix notation
D. Operator

nielit2021dec-scientistb compiler-design three-address-code

Answer key

Answer Keys

4.0.1	D	4.0.2	C	4.0.3	A	4.0.4	B	4.0.5	B
4.1.1	D	4.2.1	A	4.3.1	C	4.3.2	D	4.3.3	B
4.3.4	D	4.3.5	2	4.3.6	A	4.3.7	A	4.3.8	C
4.3.9	B	4.4.1	B	4.5.1	B	4.6.1	A	4.6.2	A
4.7.1	A	4.7.2	C	4.8.1	C	4.9.1	B	4.10.1	C
4.11.1	B	4.12.1	A	4.12.2	C	4.13.1	D	4.14.1	B
4.15.1	B	4.15.2	A	4.15.3	A	4.16.1	D	4.16.2	C
4.16.3	C	4.17.1	D	4.17.2	B	4.18.1	A	4.18.2	C
4.18.3	C	4.18.4	D	4.18.5	B	4.18.6	A	4.18.7	B
4.18.8	A	4.19.1	B	4.20.1	A	4.21.1	D	4.21.2	C
4.21.3	C	4.22.1	A						